`tests/dynamic/suites/ddos_flood_suite.sh` — operator guide

Revision: 1 **Last modified:** 2026-07-01T00:00:00Z **Status:** Authored for P10. Honest-SKIPs (`topology_unsupported`, §11.4.69) until the live `dynamic` stack lands; parse-clean + authored today.

> Companion (§11.4.18) to the in-source documentation block at the top of the script.

Overview

§11.4.169 **DDoS / load-flood** suite. It hammers the live `dynamic` stack with a sustained request flood and asserts it **degrades gracefully** rather than **collapses**: under overload the proxy may shed load (503/429/timeout) but MUST (a) keep its process alive (**Squid PID unchanged** across the flood — §11.4.108 runtime-signature), (b) **recover to normal 200** service after the flood ends, and (c) **never fail OPEN** (a flooded proxy that starts leaking / letting unauth through is the worst failure). Prefers `vegeta` / `k6` when present, else a bounded parallel-curl flood.

Prerequisites

- Source library `tests/dynamic/lib/analyzer_common.sh` + committed `tests/lib/evidence.sh`.
- `curl`, POSIX `sh`, `date`; optionally `vegeta` (preferred load tool).
- For a real run: `HELIX_DYNAMIC_STACK=1` + `HELIX_PROXY_URL`; `HELIX_SQUID_PID_CMD` to read the runtime PID signature.

Usage examples

```
# Today (no live stack): honest SKIP, exit 0:
bash tests/dynamic/suites/ddos_flood_suite.sh

# P10 live run (bounded on a shared host):
HELIX_DYNAMIC_STACK=1 HELIX_PROXY_URL=http://127.0.0.1:34128 \
```

```
FLOOD_RATE=200 FLOOD_SECS=15 FLOOD_CONC=25
      HELIX_SQUID_PID_CMD=... \
bash tests/dynamic/suites/ddos_flood_suite.sh
```

Env: `FLOOD_RATE` (req/s, default 200), `FLOOD_SECS` (default 15), `FLOOD_TARGET` (default `http://target-a.internal/`), `FLOOD_CONC` (bounded parallel fallback workers, default 25), `FLOOD_RECOVER_SECS` (settle before the recovery probe).

Edge cases

- **Live stack absent** → `topology_unsupported` SKIP, exit `0`.
- **PID changed across the flood** (crash) OR **no recovery to 200** → `FAIL`.
- **Bounded by design** — `FLOOD_CONC` caps parallelism and `nice` is used; the flood targets the stack, not the host, so the §12.6 60% host-memory ceiling is respected. Operator may lower `FLOOD_*` on shared hosts.

§11.4.115 RED_MODE polarity

`RED_MODE`	What it asserts	PASS means
`0` (default)	GREEN guard	PID stable across the flood + recovered to 200 (degraded-not-collapsed)
`1`	RED baseline	the pre-fix stack **crashes (PID changes) or does not recover** → defect reproduced

A `RED_MODE=1` run where the stack survived + recovered is a finding, not a pass.

Internal behaviour

- `#!/usr/bin/env bash`, POSIX-clean (`sh -n` + `bash -n`, §11.4.67).
- Snapshots PID before; floods via `vegeta` (writes `vegeta.report`) or bounded parallel curl workers each looping for `FLOOD_SECS` (per-worker `flood.N.codes`, tallying total / served-200 / shed-503/429); snapshots PID after; recovery probe → `flood.evidence`.
- GREEN requires `pid_before == pid_after` (PID stable) AND `recovery == 200`. Evidence dir: `qa-results/p9-harness/ddos_flood_<run-id>/`.

Related

- `tests/dynamic/lib/analyzer_common.sh` — sourced base.
- `tests/dynamic/suites/run_all_suites.sh` — orchestrates this suite.
- Constitution §11.4.169 / §11.4.85 / §11.4.108 / §11.4.69 / §11.4.115 / §12.6; design spec §13.

Last verified

2026-07-01 — run with no live stack: honest SKIP, exit `0`; `sh -n` + `bash -n` parse-clean. Live flood + recovery is exercised in **P10**.